



Micro-services - Angular

Rapport

2025 - 2026

Pokemon Drafter

Cuyala Antonin

Nichèle Pierre

Radin Matthias

Wagnier-Potier Charlyne

Sommaire

Introduction	3
Backend	4
Architecture	4
Services	5
Kubernetes	5
Frontend	6
Maquette	6
Tests Unitaires	13

Introduction

Pour un clin d'œil à la licence Pokémon, nous voulions le nommer à la manière des jeux Pokémon, opposant deux images. Notre projet s'appelle donc Pokémon Aube et Crépuscule.

Pokémon Aube et Crépuscule est une application web multijoueur inspirée de l'univers Pokémon, dans laquelle deux équipes s'affrontent : l'équipe Rouge et l'équipe Bleu. Chaque utilisateur choisit son camp lors de son inscription, constitue son équipe de Pokémon, et peut défier des adversaires de l'équipe opposée en combat en temps réel.

L'application repose sur une architecture microservices, où chaque fonctionnalité majeure est isolée dans un service indépendant : authentification, gestion des équipes, catalogue Pokémon, gestion des équipes de combat, gestion des duels, et moteur de combat. Ces services communiquent entre eux de manière asynchrone via Apache Kafka, garantissant un couplage faible et une bonne scalabilité.

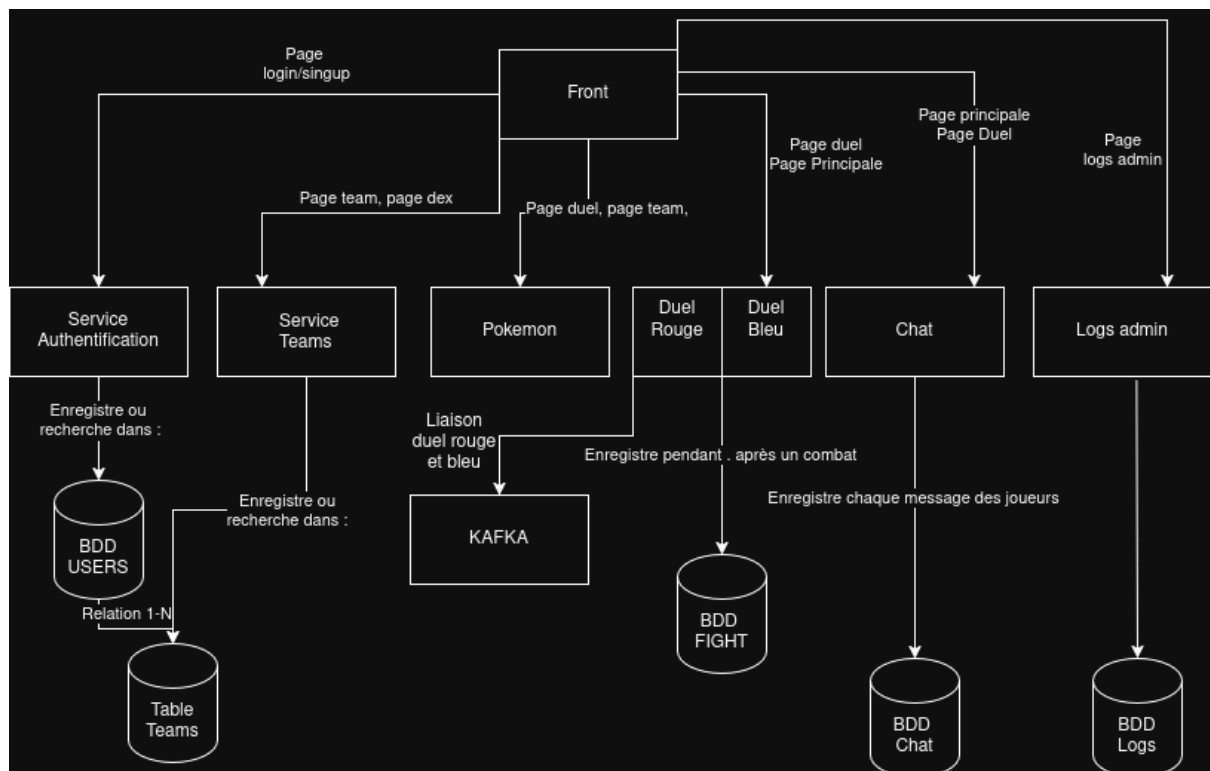
Le front-end est développé en Angular et consomme les différentes API REST exposées par les microservices. L'ensemble de l'infrastructure est conteneurisé avec Docker et orchestré via Kubernetes, permettant un déploiement reproductible et une gestion simplifiée des services.

Un grand nombre de fonctionnalités sont décrites dans le [readme.md](#). Si vous ne trouvez pas quelque chose dans ce fichier il y a de grandes chances que ce soit dans le readme.

Backend

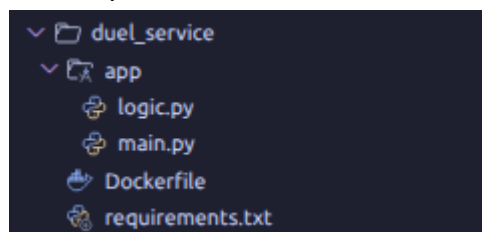
Architecture

Pour le backend, après analyse du sujet nous avons réalisé cette architecture que nous avons essayé de réaliser.



Au centre, chaque rectangle représente un service créé, avec sa bdd lorsque nécessaire.

Voici à quoi ressemble un service sur le projet :



Dans le fichier, app , nous avons les fichiers python contenant le code, les endpoints etc. Chaque service possède un Dockerfile et un requirements afin de monter le service.

Le docker-compose.yml permet de monter l'ensemble des services avec pour chacun un port attribué.

Services

Les services sont développés en Python avec le framework FastAPI. Ils s'appuient sur une base de données MySQL pour la persistance, et publient des événements sur Apache Kafka pour notifier les autres microservices des actions utilisateurs.

Le service d'authentification constitue une base solide pour l'application Pokémon Aube & Crépuscule. Il implémente les fonctionnalités essentielles d'inscription, de connexion et de gestion des sessions via JWT, tout en s'intégrant dans l'architecture microservices via Kafka. Les fonctionnalités de ce service sont login et register. Register permet à un nouvel utilisateur de créer son compte, il vérifie que l'email n'est pas déjà utilisé, hache le mot de passe avec bcrypt, crée l'utilisateur en base et publie un événement user.registered sur Kafka. Login permet à un utilisateur existant de se connecter, il vérifie les identifiants, génère un token JWT valable 60 minutes et publie un événement user.logged_in sur Kafka.

Pour finir, voici les services que nous avons :

- kafka , Ports : 9092:9092. Service kafka
- auth_service, Ports : 8000:8000. Ce service assure la création des comptes et ce qui entoure les comptes.
- pokemon-service : Ports 8002:8000. Ce service est le service de pokédex, accède à l'api pokémon.
- pokemon-team-service : Ports 8003:8000. Ce service gère la création et tout ce qui entoure les équipes pokémons.
- duel-service : Ports 8004:800. Ce service gère la logique et le déroulé des duel.
- battle-engine: Ports 8005:8000. Ce service gère l'échange entre les deux joueurs lors d'un duel
- log_service: Ports 8006:8000. Ce service gère la conservation des logs admins.

Les services ont tous des endpoints définis et l'accès aux /docs des service est plus difficile à cause du DNS, voici pour chaque service la route défini dans l'environnement et le /docs :

Service	route environnement	route docs
auth	/api/auth	/api/auth/docs
chat	/api/chat	/api/chat/docs
battle_engine	/api/battle	/api/battle/docs
pokemon_teams	/api/pokemontteams	/api/pokemontteams/docs
pokemon	/api/pokemon	/api/pokemon/docs
duel	/api/duel	/api/duel/docs
log	/api/log	/api/log/docs

Kubernetes

Docker Compose s'est révélé être un outil adapté pour l'initialisation du projet et le développement local cependant il présente des limites pour simuler un environnement de production complet (haute disponibilité, routage centralisé, auto-réparation). C'est pourquoi la suite logique de notre démarche a été de migrer l'ensemble de notre architecture vers Kubernetes .

Cette migration s'est articulée autour de quatre axes majeurs : la redéfinition des conteneurs, la mise en place d'un routage unifié, la gestion de la résilience, et l'automatisation du déploiement.

Dans notre architecture Kubernetes, chaque microservice est désormais régi par deux ressources distinctes :

Le Deployment : Il assure la gestion du cycle de vie des pods (conteneurs). Nous l'avons configuré avec la politique imagePullPolicy: Never afin d'exploiter les images Docker compilées directement dans l'environnement local du cluster, optimisant ainsi les temps de déploiement et évitant la dépendance à un registre externe.

Le Service (ClusterIP) : Sous Docker Compose, chaque service exposait un port de manière rigide sur la machine hôte (ex: 8000 pour l'Auth, 8002 pour le Pokédex). Avec Kubernetes, ces ports ne sont plus exposés à l'extérieur. Les services communiquent de manière sécurisée via un réseau privé interne en utilisant la résolution DNS native de Kubernetes (ex: le Log Service communique avec Kafka via l'adresse interne kafka-svc:9092).

Afin d'offrir un point d'entrée unique et propre pour notre application, nous avons implémenté un Ingress Controller (Nginx), couplé à une configuration DNS locale.

L'application n'est plus accessible via localhost mais via un nom de domaine dédié : `http://pokemon.aube.crepuscule`. Au lieu de requêter différents ports, le client frontend n'interroge plus qu'une seule URL. L'Ingress analyse le chemin de la requête HTTP et la redirige vers le bon micro service en interne (par exemple, le chemin `/api/auth` redirige vers le pod `auth-service`, et `/api/pokemon` vers le `pokemon-service`).

Nos APIs développées sous FastAPI démarrent presque instantanément, tandis que Apache Kafka et le gestionnaire Zookeeper ,nécessitent un temps plus long.

Si un service démarre et échoue à se connecter à Kafka, il s'arrête en erreur. Kubernetes détecte ce crash et redémarre automatiquement le pod jusqu'à ce que la connexion soit établie avec succès.

Pour les tâches uniques, comme la création des topics Kafka, nous avons utilisé des ressources de type Job. Contrairement aux Déploiements qui s'exécutent en continu, le Job exécute son script d'initialisation puis adopte le statut Completed, libérant ainsi les ressources du cluster.

L'ensemble du processus Kubernetes a été automatisé via un script Bash d'installation ([start.sh](#)). Ce script se charge de manière transparente de :

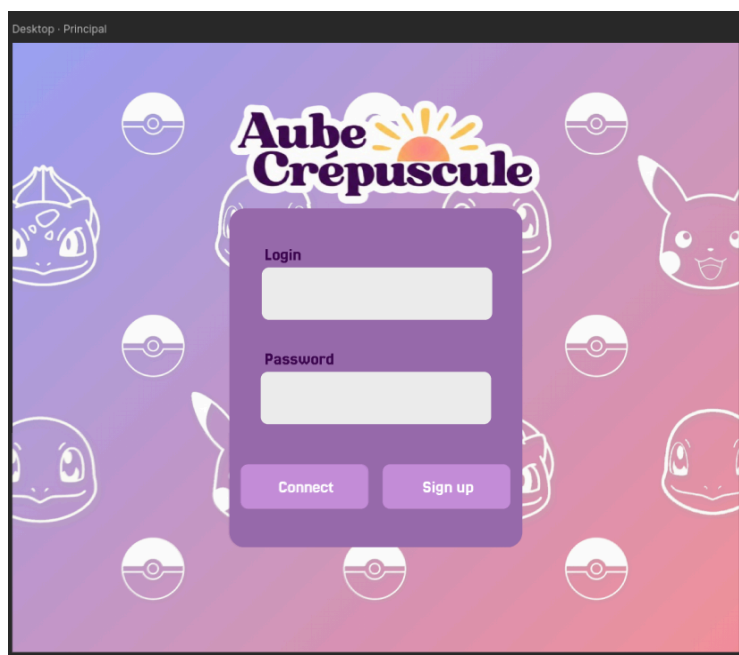
- Démarrer le cluster Minikube et activer l'Ingress.
- Construire à la volée l'ensemble des 9 images Docker des microservices au sein du cluster.
- Appliquer l'ensemble des manifestes YAML (`kubectl apply`).
- Configurer automatiquement le fichier `hosts` de la machine pour la résolution du nom de domaine.

Frontend

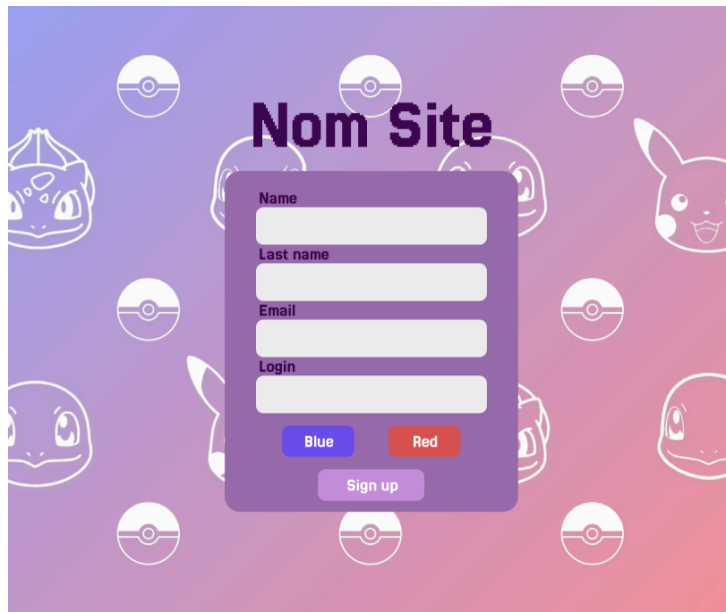
Maquette

Pour le frontend, nous avons décidé de ne pas partir têtes baissées dans le code mais nous voulions avoir une maquette proche de ce que nous voulions réaliser. Pour faire la maquette nous sommes allés sur figma. Notre site reprend beaucoup de détails de la licence Pokémon.

Maquette page de login :



Maquette page de création de compte:



Maquette de page de création de compte Pokémon. Le fond est un dégradé de violet à rose, parsemé de motifs de Pokémon (Poliwhirl, Pikachu, Squirtle) et de Pokéballs. Au centre, un formulaire blanc est superposé.

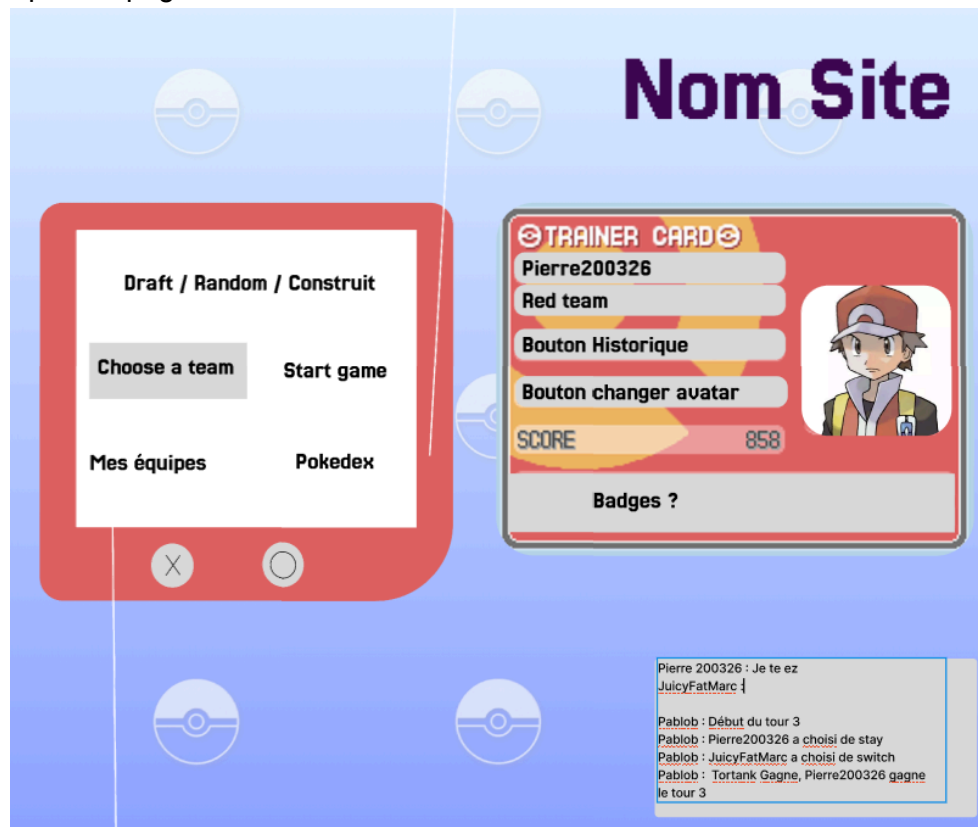
Nom Site

Le formulaire contient les champs suivants :

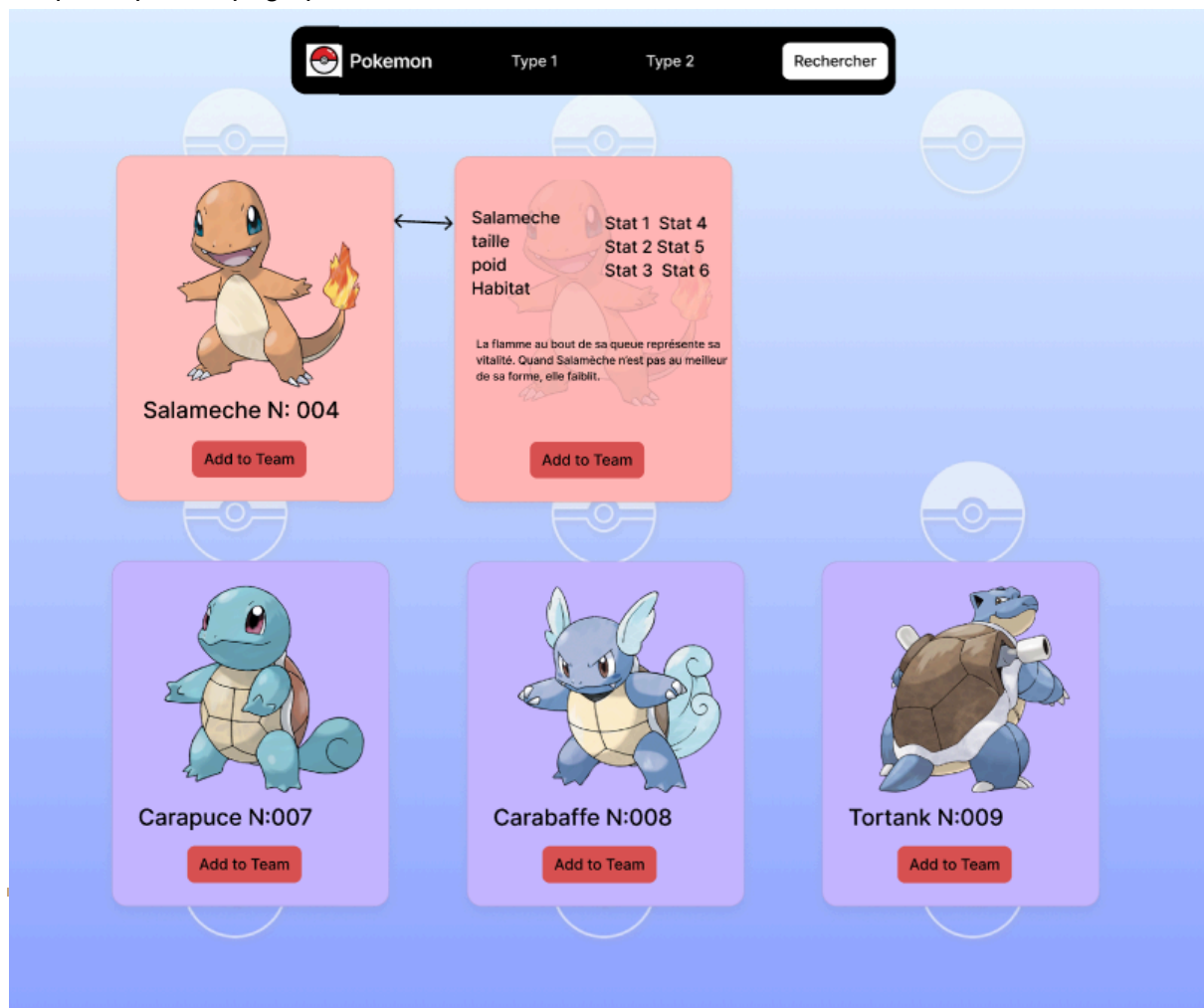
- Name
- Last name
- Email
- Login

En dessous des champs, il y a deux boutons : **Blue** et **Red**. En bas du formulaire, un bouton **Sign up** est visible.

Maquette pour la page home:



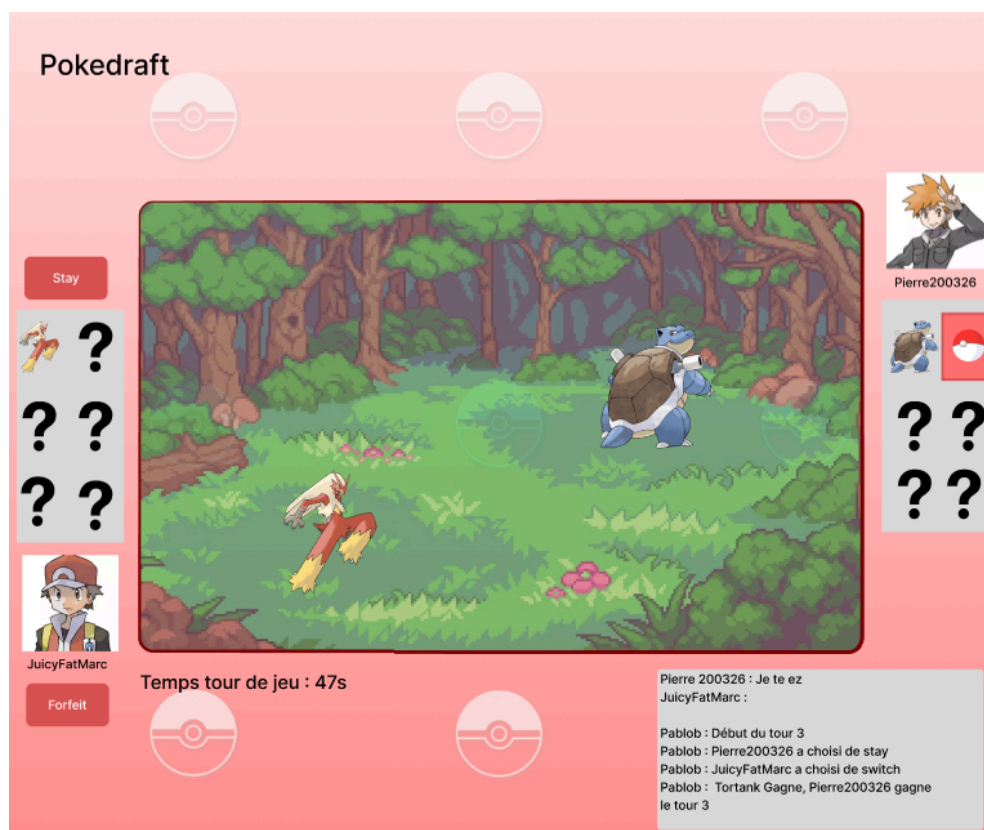
Maquette pour la page pokédex :



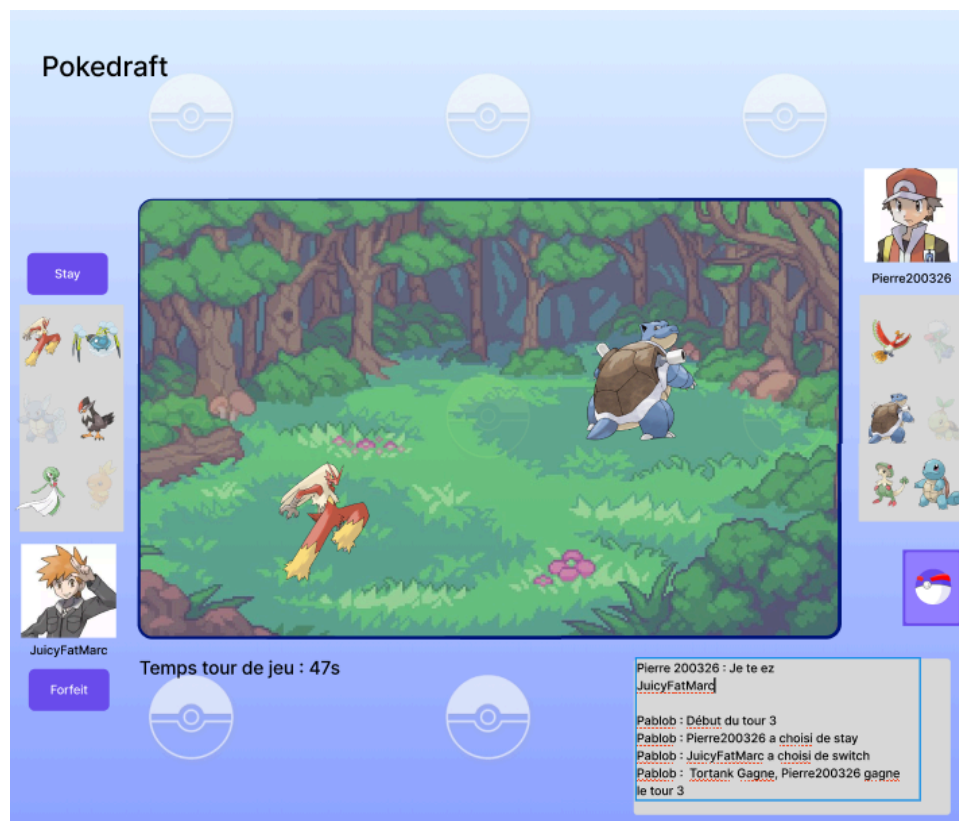
Maquette pour la page team pokémon :



Maquette pour la page de duel hasard :



Maquette pour la page de duel construit :



Maquette pour la page draft :

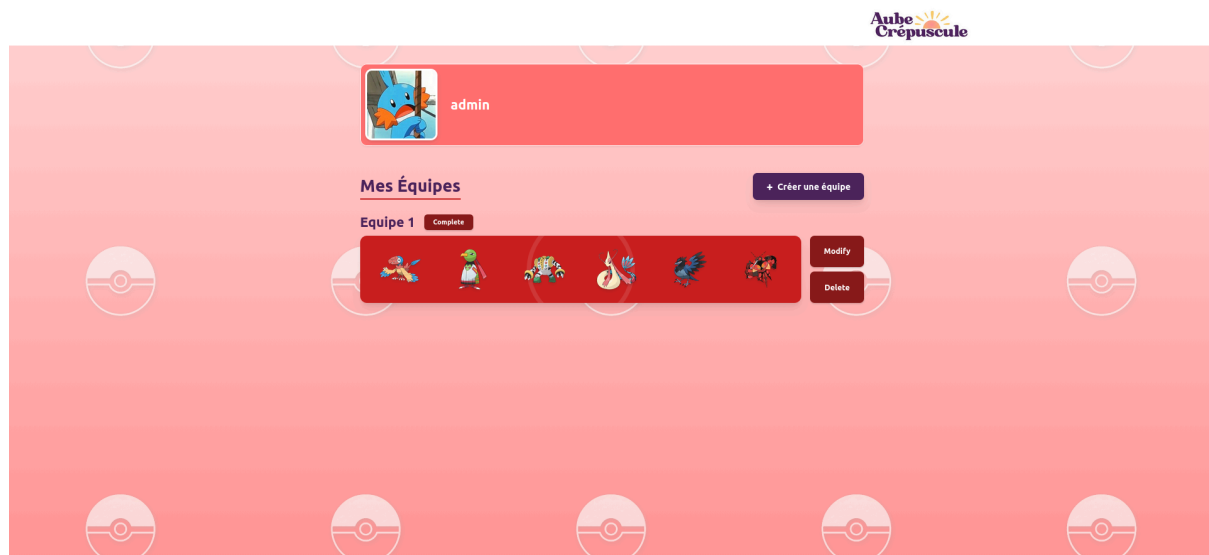


Page de connexion et d'inscription :

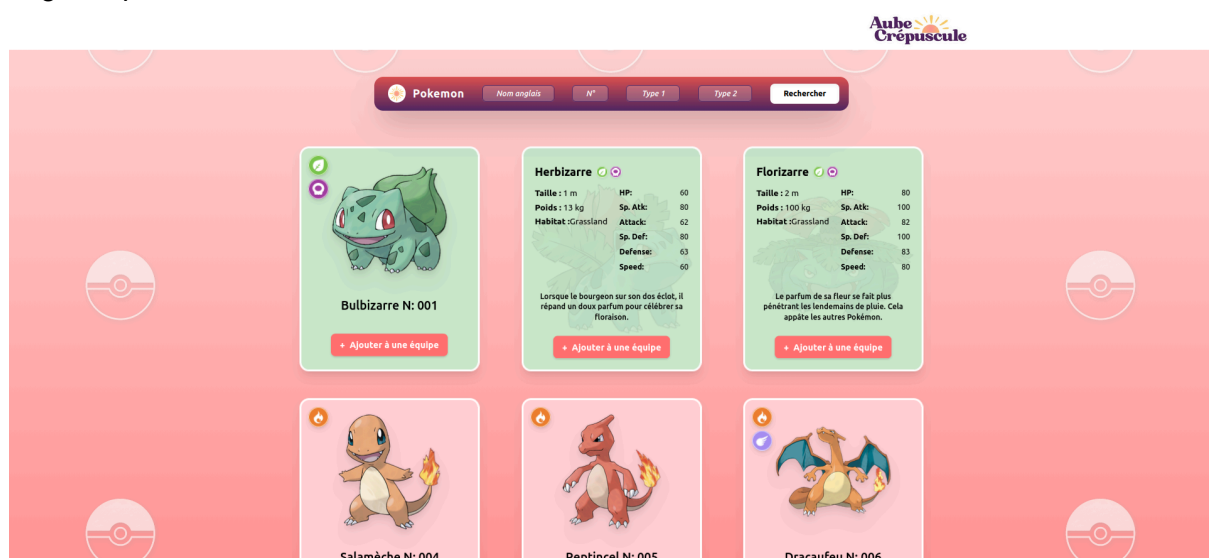
The background is a vertical gradient from light purple at the top to pink at the bottom. It features several white-outlined illustrations of Pokémon heads: Bulbasaur, Squirtle, Charmander, and Pikachu. There are also two Poké Balls floating in the scene.

Page Home avec redirection vers les différents

Page des équipes :



Page du pokédex :



Le chat est disponible à deux endroits dans l'application. Sur la page Home, il permet aux joueurs connectés d'échanger dans un salon global. Chaque message affiche le pseudo du joueur, avec une couleur liée à son équipe. Le joueur peut aussi activer une option pour ne lire que les messages de son équipe et rendre ses propres messages visibles uniquement par les membres de sa couleur.

Sur la page Duel, le chat est lié au combat en cours. Il contient un onglet Journal du combat, qui affiche les événements importants de la partie, et un onglet Chat joueur, qui permet aux adversaires d'échanger pendant le duel. Ce chat reste propre au match et n'est pas affecté par le filtre d'équipe de la page Home.

Page de combat hasard :

TOUR 3

Temps restant : 78s

t

Scorplane

Axoloto

JOURNAL DU COMBAT

CHAT JOUEUR

Unquin (Points 1.0) VS Jaminor (Points 2.0)
Jodifon l'emporte ! Unquin est mis KO !
Pablo : --- DÉBUT DU TOUR 3 ---
admin a fait : SWITCH
t a fait : SWITCH
Axoloto (Points 2.0) VS Scorplane (Points 2.0)
Égalité ! Les deux Pokémon sont ko.

admin

STAY

FORFAIT

Page de combat construit :

TOUR 1

Temps restant : 61s

t

Denticrise

Arkapti

JOURNAL DU COMBAT

CHAT JOUEUR

Pablo : Les équipes personnalisées sont validées ! Le combat commence.

admin

STAY

FORFAIT

Page de combat draft :

Aube Crépuscule

PHASE DE DRAFT

t

À VOTRE TOUR DE CHOISIR !

JOURNAL DU COMBAT

CHAT JOUEUR

Pablo : Au tour du joueur Rouge de choisir !
Pablo : Au tour du joueur Bleu de choisir !
Pablo : Au tour du joueur Rouge de choisir !

admin

STAY

FORFAIT

Tests Unitaires

Les tests sont écrits avec le framework Jasmine et exécutés par Karma dans un navigateur Chrome headless. Ils peuvent être lancés à tout moment avec la commande `ng test`, sans avoir besoin de démarrer les serveurs back-end, Kafka ou la base de données. Aucun test n'appelle réellement le serveur. Les requêtes HTTP sont interceptées par `HttpTestingController`, un utilitaire Angular qui simule les réponses sans passer par le réseau. Les services Angular sont remplacés par des spies Jasmine qui imitent leur comportement.

Fichier Testé	Type	Aspects Couverts
login.spec.ts	Composant	Valeurs initiales, connexion, gestion des erreurs
register.spec.ts	Composant	Validation des champs, sélection d'équipe, inscription
auth.spec.ts	Service	Register, login, getMe, updateProfil, token, localStorage, logout
pokemon.spec.ts	Service	Recherche avec filtres, pagination
pokemon-team.spec.ts	Service	CRUD équipes, méthodes HTTP
pokemon-teams.spec.ts	Composant	Chargement profil, redirect si non-connecté
dex.spec.ts	Composant	Recherche Pokémon, pagination, ajout en équipe
home.spec.ts	Composant	Profil, modes de jeu, avatar, logout
duel.spec.ts	Composant	Etats du jeu, action combat, timer, fin de match, aura